

Grundlagen der IT

Teil 2:

Zahlensysteme und Konvertierungen

Binäre Arithmetik

Informationstheorie

Codierungen

Bernd Schöner

DHBW Mannheim

Darstellung von Zahlen

- Für das praktische Rechnen verwendet man dem Problem angepasste Ziffernsysteme. Für Menschen am geläufigsten ist das Dezimalsystem. Für die digitale Datenverarbeitung sind jedoch Ziffernsysteme günstiger, die dem Umstand Rechnung tragen, dass für die Darstellung von Zahlen in digitalen Rechenanlagen nur die beiden Ziffern 0 und 1 verwendet werden.
- Als elementare Maßeinheit für Informationen dient das Bit. Eine Nachricht (z.B. ein Text od. eine Zahl) hat einen Informationsgehalt von n Bit.
- Der Informationsgehalt einer Nachricht ist eine Größe, die angibt, wie viel Information in dieser Nachricht übertragen wurde.
- Er bezeichnet die minimale Anzahl von Bits, die benötigt werden, um ein Zeichen (also eine Information) darzustellen oder zu übertragen.

Zahlensysteme

- Ein Zahlensystem legt fest, wie eine Zahl dargestellt wird, Unter Verwendung einer kleinen Anzahl von Ziffern.
- In einem leistungsfähigen Zahlensystem können die Ziffern, z.B. 0, 1, 2, ..., 9, ergänzt sein durch Vorzeichen, Dezimaltrennzeichen und Tausendertrennzeichen.
- Die Ziffern werden je nach ihrer Stelle unterschiedlich bewertet (Einer, Zehner, Hunderte, ...)
- Unter einem Zahlensystem zur Basis b verstehen wir ein Stellenwertsystem, bei dem Werte aus der Addition einzelner Ziffern aus der Menge $\{0, 1, \dots, b-1\}$ multipliziert mit Potenzen der Basis b dargestellt werden: Die n -stellige Zahl mit m Nachkommastellen hat den Wert:

$$\sum_{k=-m}^{n-1} d_k b^k = d_{n-1} b^{n-1} + \dots + d_0 + d_{-1} b^{-1} + \dots + d_{-m} b^{-m}$$

- Bei Zahlensystemen mit einer Basis $b > 10$ werden die Buchstaben A-Z für Ziffern mit Wert ab 10 verwendet.
- Am häufigsten kommen für diesen Zweck (als Beispiele)
 - das Binärsystem (101_{bin})
 - das Hexadezimalsystem ($2A_{\text{hex}}$)
 - bisweilen auch das Oktalsystem (37_{okt}) zur Anwendung.
- Werden Zahlen ohne einen Index zum Zahlensystem angegeben, so ist hier die Dezimaldarstellung gemeint.

Überblick über reelle Zahlen

reelle Zahlen ($\mathbb{R}$)						
rationale Zahlen ($\mathbb{Q}$)					irrationale Zahlen	
ganze Zahlen ($\mathbb{Z}$)			Bruchzahlen			
positive ganze Zahlen ($\mathbb{N} = \mathbb{Z}^+$)	Null	negative ganze Zahlen ($\mathbb{Z}^-$)	positive	negative	positive	negative

Dezimalsystem/1

- Im Dezimalsystem (Zehnersystem) wird eine natürliche Zahl z als Summe von Potenzen zur Basis $B = 10$ dargestellt (**Summenformel**):

$$z = a_n 10^n + a_{n-1} 10^{n-1} + \dots + a_2 10^2 + a_1 10^1 + a_0 10^0$$

$$a_0, a_1, \dots, a_n \in \{0, 1, \dots, 9\}$$

$$\text{Basis : } 10_{\text{dez}}$$

$$n \in \mathbb{N}$$

Beispiel : $2789 = 2 \times 10^3 + 7 \times 10^2 + 8 \times 10^1 + 9 \times 10^0$

Dezimalsystem/2

- Erweitert man dieses Konzept um negative Exponenten, so lassen sich auch Dezimalbrüche, d. h. rationale Zahlen r darstellen:

$$r = a_n 10^n + \dots + a_1 10^1 + a_0 10^0 + a_{-1} 10^{-1} + a_{-2} 10^{-2} + \dots + a_{-m} 10^{-m}$$

$$a_0, a_1, \dots, a_n \in \{0, 1..9\},$$

Basis : 10_{dez}

$$r \in \mathbb{Q}$$

- Eine rationale Zahl hat in obiger Notation also $n+1$ Vorkommastellen und m Nachkommastellen.

Beispiel: Die Zahl 257,34 lautet in der oben definierten Darstellung:

$$257,34 = 2 \cdot 10^2 + 5 \cdot 10^1 + 7 \cdot 10^0 + 3 \cdot 10^{-1} + 4 \cdot 10^{-2}$$

Dezimalsystem/3

- Unendliche, d. h. nicht abbrechender Dezimalbruch oder eine reelle Zahl mit unendlich vielen Stellen:

$$i = a_n 10^n + \dots + a_1 10^1 + a_0 10^0 + a_{-1} 10^{-1} + a_{-2} 10^{-2} + \dots + a_{-\infty} 10^{-\infty}$$

$$a_0, a_1, \dots, a_n \in \{0, 1..9\}$$

Basis : 10_{dez}

$$i \in \mathbb{R} \setminus \mathbb{Q}$$

$$\text{Beispiel: } \pi = 3.141\dots = 3 \times 10^0 + 1 \times 10^{-1} + 4 \times 10^{-2} + 1 \times 10^{-3} \dots$$

Bemerkung: $\mathbb{R} \setminus \mathbb{Q}$ steht für die Menge der irrationalen Zahlen

Dualsystem

- Auf Grund der Repräsentation von Daten in DV-Anlagen durch die beiden Zustände „0“ und „1“ bietet sich in diesem Bereich das Dualsystem (auch Zweiersystem oder Binärsystem) an. Es arbeitet mit der Basis $B = 2$ und den beiden Grundziffern $G = \{0,1\}$

$$b = \underbrace{a_n 2^n + a_{n-1} 2^{n-1} + \dots + a_1 2^1 + a_0 2^0}_{r \in \mathbb{Z}}$$

$$\underbrace{a_{-1} 2^{-1} + a_{-2} 2^{-2} + \dots + a_{-m} 2^{-m}}_{i \in \mathbb{Q}} + \dots + a_{-\infty} 2^{-\infty}$$

$$\underbrace{\hspace{10em}}_{i \in \mathbb{R} \setminus \mathbb{Q}}$$

$$a_n, \dots, a_1, a_0, \dots, a_{-m}, a_{-\infty} \in \{0, 1\}$$

Basis : 2_{dez}

Beispiel: $10010 = 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 18_{\text{dez}}$

Hexadezimalsystem

- Eine noch kompaktere Zahldarstellung ergibt sich, wenn man jeweils vier Binärziffern zu einer Hexadezimalziffer zusammenzieht. Dieses Ziffernsystem ist u. a. deshalb sehr praktisch, weil die mit einem Byte codierbaren Zahlen (8 Bit) gerade mit zweistelligen Hexadezimalzahlen geschrieben werden können.

$$h = \underline{a_n 16^n + a_{n-1} 16^{n-1} + \dots + a_1 16^1 + a_0 16^0}$$

$$h \in \mathbb{Z}$$

$$+ \underline{a_{-1} 16^{-1} + a_{-2} 16^{-2} + \dots + a_{-m} 16^{-m}} + \dots + a_{-\infty} 16^{-\infty}$$

$$\underline{h \in \mathbb{Q}}$$

$$H \in \mathbb{R} \setminus \mathbb{Q}$$

$$a_0, a_1, \dots, a_n, a_{-m}, a_{-\infty} \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$$

Basis : 16_{dez}

Beispiel: $0x1C4F = 1 \times 16^3 + C \times 16^2 + 4 \times 16^1 + F \times 16^0 = 7247_{\text{dez}}, \text{ bzw. } 0001\ 1100\ 0100\ 1111_{\text{bin}}$

Hexadezimalsystem

- Die Zahlenwerte von Brüchen ergeben sich durch Multiplizieren der Stellenwerte mit B^{-k} , wobei B die Basis des Ziffernsystems ist und k die Position, also z. B. 1 für die erste und 2 für die zweite Nachkommastelle.
- Im Hexadezimalsystem entspricht der ersten Nachkommastelle also der dezimale Zahlenwert $16^{-1} = 0,0625$, der zweiten $16^{-2} = 0,00390625$ usw.
 - $1/16 = 0,0625$
 - $1/16^{-2} = 1/256 = 0,00390625$
- Im Binärsystem hat die erste Nachkommastelle den Wert $2^{-1} = 0,5$, die zweite den Wert $2^{-2} = 0,25$.
 - $\frac{1}{2} = 0,5$
 - $1/2^{-2} = \frac{1}{4} = 0,25$

Zahlendarstellungen in Dez, Bin, Hex

Dez	Bin	Hex	Dez	Bin	Hex
0	0000	0x0	8	1000	0x8
1	0001	0x1	9	1001	0x9
2	0010	0x2	10	1010	0xA
3	0011	0x3	11	1011	0xB
4	0100	0x4	12	1100	0xC
6	0101	0x5	13	1101	0xD
6	0110	0x6	14	1110	0xE
7	0111	0x7	15	1111	0xF

0 - 15_{dez} in Binär-, Dezimal- & Hexadezimaldarstellung

Bits and Bytes

- Ein **Bit** („binary digit“) bezeichnet die **kleinste Informations- und Zähleinheit in einem Computer in Form einer einstelligen Binärzahl**. Es kann die Werte 0_{bin} und 1_{bin} annehmen. Eine achtstellige Binärzahl wird als ein Byte bezeichnet.
- Ein **Byte** lässt sich damit entweder mit einer **8-stelligen Binärzahl**, mit 3-stelligen Oktal- oder Dezimalzahlen oder mit 2-stelligen Hexadezimalzahlen darstellen.

Bits werden in 8er Blöcken zu Bytes zusammengefasst,

1010 1100	1100 1110	1010 0011	1111 1010	...
1.Byte	2.Byte	3.Byte	4.Byte	

Stellenwertigkeit $2^0 - 2^7$ in Bin, Dez, Hex.

	Dez	Bin	Hex
2^0	1	0000 0001	0x01
2^1	2	0000 0010	0x02
2^2	4	0000 0100	0x04
2^3	8	0000 1000	0x08
2^4	16	0001 0000	0x10
2^5	32	0010 0000	0x20
2^6	64	0100 0000	0x40
2^7	128	1000 0000	0x80

Beispiel: Was ist 1111 1111 in Dez und Hex?

Konvertierung Bin, Okt, Hex -> Dez

Die Umrechnung von Binär-, Oktal- oder Hexadezimalzahlen zu Dezimalzahlen erfolgt durch Addition der Produkte der Ziffern mit ihrem Stellenwert.

Die Binärzahl 1011,01 hat die dezimale Darstellung

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \cdot 2^{-2} = 8 + 2 + 1 + 0,25 = 11,25$$

Die Oktalzahl 64_{okt} hat die dezimale und binäre Darstellung

$$6 \times 8^1 + 4 \times 8^0 = 48 + 4 = 52_{\text{dez}} \text{ und } 110\ 100_{\text{bin}}$$

Die Hexadezimalzahl $x = F8_{\text{hex}}$ hat die dezimale und binäre Darstellung

$$15 \times 16^1 + 8 \times 16^0 = 240 + 8 = 248 \text{ und } 1111\ 1000_{\text{bin}}$$

Konvertierung Dez -> Bin durch Divisionsmethode

Man dividiert die umzuwandelnde Dezimalzahl durch die größte Potenz von 2, die kleiner ist als die Dezimalzahl und notiert als erste (höchstwertige) Binärstelle eine 1. Das Ergebnis wird nun durch die **nächst kleinere Potenz** von 2 dividiert; das Resultat, also 0 oder 1, gibt die nächste Binärstelle an, usw. bis schließlich nach der Division durch $2^0 = 1$ das Verfahren abbricht. Auf analoge Weise kann man eine in irgendeinem Zahlensystem angegebene Zahl in eine beliebige andere Basis umwandeln.

$$116 : 64 = 1$$

$$\underline{-64}$$

$$52 : 32 = 1$$

$$\underline{-32}$$

$$20 : 16 = 1$$

$$\underline{-16}$$

$$4 : 8 = 0$$

$$\underline{-0}$$

$$4 : 4 = 1$$

$$\underline{-4}$$

$$0 : 2 = 0$$

$$\underline{-0}$$

$$0 : 1 = 0$$

$$116_{\text{dez}} = 111\,0100_{\text{bin}}$$

Konvertierung Dez -> Bin und Hex durch Restwertmethode

Die fortgesetzte Division einer Dezimalzahl durch Basis B , der Divisionsrest liefert die Koeffizienten a_0 bis a_n für die Darstellung dieser Zahl zur Basis B .

Beispiel: Es soll die Zahl 10172_{dez} mit der Restwertmethode in duale und hexadezimale Schreibweise umgewandelt werden.

Die Umwandlung in eine Dualzahl folgt aus der fortgesetzten Division durch 2:

$10172 : 2 = 5086$ Rest 0	$317 : 2 = 158$ Rest 1	$9 : 2 = 4$ Rest 1
$5086 : 2 = 2543$ Rest 0	$158 : 2 = 79$ Rest 0	$4 : 2 = 2$ Rest 0
$2543 : 2 = 1271$ Rest 1	$79 : 2 = 39$ Rest 1	$2 : 2 = 1$ Rest 0
$1271 : 2 = 635$ Rest 1	$39 : 2 = 19$ Rest 1	$1 : 2 = 0$ Rest 1
$635 : 2 = 317$ Rest 1	$19 : 2 = 9$ Rest 1	

Ergebnis: $10172_{\text{dez}} = 10011110111100_{\text{bin}}$ (Achtung Reihenfolge  von hinten nach vorne)

Fortgesetzte Division durch 16 liefert für die Umwandlung in eine Hexadezimalzahl:

$10172 : 16 = 635$ Rest 12	(= C)
$635 : 16 = 39$ Rest 11	(= B)
$39 : 16 = 2$ Rest	(= 7)
$2 : 16 = 0$ Rest	(= 2)

Ergebnis: $10172_{\text{dez}} = 27BC_{\text{hex}}$ (Achtung Reihenfolge von hinten nach vorne)

Konvertierung Dezimalbrüche -> Bin und Hex durch Restwertmethode

Es soll die Dezimalzahl 39,6875 als Dualzahl dargestellt werden.

Vorkommastellen, wie gehabt: Ergebnis: $39_{\text{dez}} = 100111_{\text{bin}}$.

Für die Konversion der Nachkommastellen, erhält man die Koeffizienten (= Ziffern) $a_{-1}, a_{-2}, \dots$ bei fortgesetzter Multiplikation mit der Basis als den ganzzahligen Teil (d. h. die Vorkommastellen des Multiplikationsergebnisses) erhält. Man lässt für den folgenden Schritt die Vorkommastellen weg und setzt dieses Verfahren fort, bis die Multiplikation eine ganze Zahl ergibt oder bis im Falle eines nicht abbrechenden Dezimalbruchs die gewünschte Genauigkeit erreicht ist.

$$0,6875 \cdot 2 = 1,375 \quad 1 \text{ abspalten}$$

$$0,3750 \cdot 2 = 0,750 \quad 0 \text{ abspalten}$$

$$0,7500 \cdot 2 = 1,500 \quad 1 \text{ abspalten}$$

$$0,5000 \cdot 2 = 1,000 \quad 1 \text{ abspalten (fertig, Ergebnis ganzzahlig)}$$

Ergebnis für die Nachkommastellen: $0,6875_{\text{dez}} = 0,1011_{\text{bin}}$.

Insgesamt hat man also: $39,6875_{\text{dez}} = 100111,1011_{\text{bin}} = 27,B_{\text{hex}}$.

Im Hexadezimalsystem ist die Rechnung viel kürzer in nur einem Schritt durchführbar:

$$0,6875 \cdot 16 = 11,000 \quad 11=B \text{ abspalten}$$

Umwandlung in Hexadezimalzahlen durch Zusammenfassen von Binärstellen

$$97_{\text{dez}} = \underbrace{0110}_6 \underbrace{0001}_1 = 0x61$$

$$531_{\text{dez}} = \underbrace{0010}_2 \underbrace{0001}_1 \underbrace{0011}_3 = 0x213$$

$$4003_{\text{dez}} = \underbrace{1111}_F \underbrace{1010}_A \underbrace{0011}_3 = 0xFA3$$

- Bei Bedarf werden führende Nullen zugefügt.

Umwandlung von Hexadezimalzahlen in Binärzahlen und Dezimal

$$2E4_{\text{hex}} = \underbrace{0010}_{\text{2}} \underbrace{1110}_{\text{E}} \underbrace{0100}_{\text{4}}_{\text{bin}} = 1011100100_{\text{bin}}$$

Führende Nullen werden unterdrückt.

Übungsaufgabe:

$$1011100100_{\text{bin}} = ??_{\text{dez}}$$

Zahlensysteme mit anderer Basis

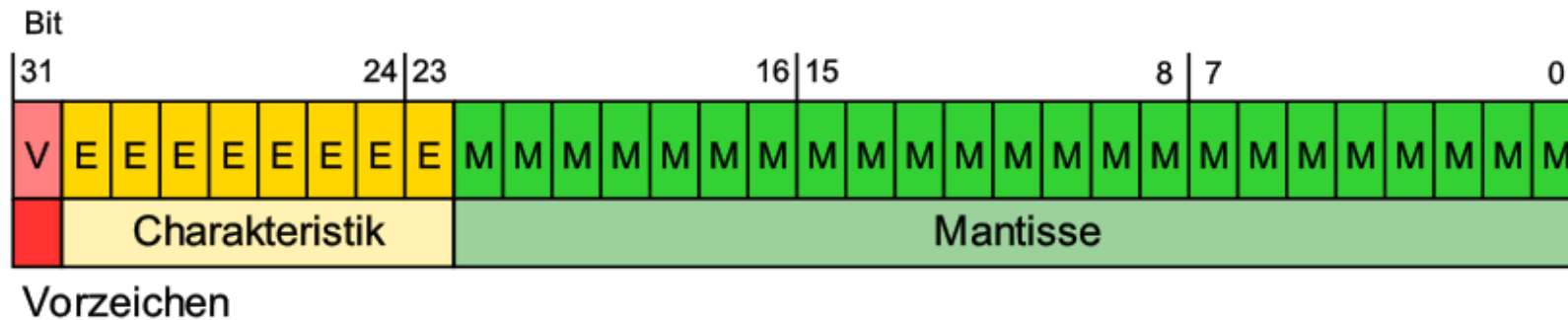
- Es gelten die vorgestellten Verfahren.
- **Beispiel:**
 - $28_{\text{dez}} = ??_5$
 - Restwertmethode:
 - $28 : 5 = 5 \text{ Rest } 3$
 - $5 : 5 = 1 \text{ Rest } 0$
 - Ergebnis: 103_5

Gleitkommazahlen

- Für die Darstellung sehr kleiner oder sehr großer Zahlen sind Festkommazahlen nicht geeignet. Günstiger und in der Computertechnik seit Zuse üblich sind Gleitkommazahlen in halblogarithmischer Darstellung.
- In Gleitkommadarstellung schreibt man Zahlen in der Form $m \cdot b^e$. Dabei ist die Mantisse m eine Festkommazahl und b^e ein Faktor mit Basis b und Exponent e . Als Mantisse m bezeichnet man die Ziffernstellen einer Gleitkommazahl.
- **Beispiele:** Dezimale Gleitkommazahlen mit der Basis $b = 10$:
 - $0,000112 = 1,12 \cdot 10^{-4}$, $7123458 = 7,123458 \cdot 10^6$, $-24,317 = -2,4317 \cdot 10^1$.
- Darstellung in **Normalform**: man schreibt die Mantisse m so, dass genau eine Stelle vor dem Dezimalpunkt eine von Null verschiedene Ziffer ist.
- Die Genauigkeit der Zahldarstellung hängt offenbar von der Stellenzahl der Mantisse ab, der darstellbare Zahlenbereich von der Basis und vor allem vom Exponenten.

Gleitkommazahlen nach IEEE 754 Standard

- Gleitkommazahlen mit Basis $b = 2$.
- Nach dem Standard3 IEEE 754 ist eine binäre Gleitkommazahl z mit Vorzeichenbit s , binärer Mantisse $1, f$ (in Normalform) und Exponent e wie folgt definiert:



- *Normalform*: genau eine Stelle vor dem Dezimalpunkt, die eine von Null verschiedene Ziffer ist
- Exponent wird nicht direkt als e , sondern in Form der sog. Charakteristik $c = e+B$ gespeichert wird. Zum tatsächlichen Exponenten e wird also eine Verschiebung (Bias) B addiert, die so gewählt ist, dass der Nullpunkt für e in die Mitte des zur Verfügung stehenden Wertebereichs $[0, 2B+1]$ verschoben wird. Auf diese Weise können Exponenten zwischen $e = -B$ (entsprechend $c = 0$) und $e = B+1$ (entsprechend $c = 2B+1$) dargestellt werden
- Bei einer kurzen Gleitkommazahl werden 32 Bit verwendet, wobei 8 Bit für die Charakteristik $c = e+127$ mit Wertebereich $[0, 255]$ und Bias $B = 127$ zur Verfügung stehen.

Beispiel: Umwandlung Dez in Gleitkomma

1. Die Dezimalzahl wird in eine Binärzahl umgewandelt, ggf. mit Nachkommastellen.
2. Das Komma wird so weit verschoben, bis die Normalform erreicht ist. Bei Verschiebung um je eine Stelle nach links oder rechts wird der Exponent e der Basis 2 um eins erhöht bzw. um eins erniedrigt.
3. Das Vorzeichen s der Zahl (positiv: 0, negativ: 1) wird in das MSB (Most Significant Bit) des ersten Byte geschrieben.
4. Zum Exponenten e wird 127 addiert, das Ergebnis $c = e+127$ wird in binäre Form mit 8 Stellen umgewandelt. Ist der Exponent positiv, so hat das führende Bit von c den Wert 1, sonst hat es den Wert 0.
5. In die Bytes 2 (anschließend an das bereits für den Exponenten verwendete MSB), 3 und 4
6. werden schließlich die *Nachkommastellen* $f_0 f_1 \dots f_{22}$ der Mantisse eingefügt.

Beispiel: 148,625 ist in eine binäre Gleitkommazahl umzuwandeln.

1. Schritt: $148,625_{\text{dez}} = 10010100,101_{\text{bin}}$
2. Schritt: $10010100,101 = 1,0010100101 \cdot 2^7$, Normalform ist erreicht, der Exponent ist $e = 7$
3. Schritt: Das Vorzeichen der Mantisse ist positiv, das MSB lautet also $s = 0$
4. Schritt: Charakteristik: $c = e+127 = 134_{\text{dez}} = 10000110_{\text{bin}}$
5. Schritt: Ergebnis: $0100\ 0011\ 0001\ 0100\ 1010\ 0000\ 0000\ 0000_{\text{bin}} = 4314A000_{\text{hex}}$
Byte 1 Byte 2 Byte 3 Byte 4

Binäre Arithmetik

- Die Rechenregeln für Binärzahlen sind ganz analog zu den Rechenregeln für Dezimalzahlen definiert.
- Es genügt, sich auf die binäre Addition, Subtraktion, Multiplikation, Division und die logischen Operationen zu beschränken.

Übersicht der Grundrechenarten und logischen Operationen

Operation	Operator	Log. OP
Addition	+	
Subtraktion	-	
Multiplikation	X	
Division	:	
Kontravalenz	$\underline{\vee}$	XOR
Disjunktion	$\vee$	OR
Konjunktion	$\wedge$	AND
Inversion	$\neg$	NOT

Anmerkung: Bei der Verwendung von „ $\vee$ “ für das logische Oder wird hingegen „ $\underline{\vee}$ “ für das Exklusiv-Oder verwendet.

Binäre Arithmetik, Wahrheitstabellen

- $\text{XOR } b = (a \wedge \neg b) \vee (\neg a \wedge b)$
- NOT: $\neg 1 = 0$ $\neg 0 = 1$

A	B	$A \vee B$	$A \wedge B$	$A \nabla B$
1	1	1	1	0
1	0	1	0	1
0	1	1	0	1
0	0	0	0	0

- **Beispiel:** Logische Verknüpfungen werden stellenweise ausgeführt:

10011	10011	
$\vee$ 10101	$\wedge$ 10101	$\neg$ 10101
= 10111	= 10001	= 01010

Binäre Arithmetik, Addition

- Die Rechenregeln für die binäre Addition zweier Binärziffern lauten:
 - $0+0 = 0$, $0+1 = 1$, $1+0 = 1$, $1+1 = 0$ Übertrag 1 .

- **Beispiel:** Binäre Addition

Die Aufgabe $11+14 = 25$ soll in binärer Arithmetik gelöst werden:

$$\begin{array}{r} 1011 \\ + 1110 \\ \hline 111 \quad \text{Übertrag} \\ = 11001 \quad \text{Ergebnis} \end{array}$$

Die Aufgabe $151,875+27,625 = 179,5$ soll in binärer Arithmetik gelöst werden:

$$\begin{array}{r} 10010111.111 \\ + 11011.101 \\ \hline 111111 \ 11 \quad \text{Übertrag} \\ = 10110011.100 \quad \text{Ergebnis} \end{array}$$

Binäre Arithmetik, Subtraktion

- Die Rechenregeln für die binäre Subtraktion zweier Binärziffern lauten:
 - $0-0 = 0$, $1-1 = 0$, $1-0 = 1$, $0-1 = 1$ Übertrag -1 .

- **Beispiel:** Direkte Binäre Subtraktion

Die Aufgabe $13-11 = 2$ soll in binärer Arithmetik gelöst werden:

$$\begin{array}{r} 1101 \\ - 1011 \\ \hline -1 \quad \text{Übertrag} \\ = 0010 \quad \text{Ergebnis} \end{array}$$

Binäre Arithmetik, Multiplikation

- Die Rechenregeln für die binäre Multiplikation entsprechen der logischen UND-Verknüpfung zweier
- Binärziffern, sie lauten:
- $0 \cdot 0 = 0$, $0 \cdot 1 = 0$, $1 \cdot 0 = 0$, $1 \cdot 1 = 1$.
- Die Multiplikation mehrstelliger Zahlen wird (wie von der Multiplikation im Zehnersystem gewohnt) auf die Multiplikation des Multiplikanden mit den einzelnen Stellen des Multiplikators und stellenrichtige Addition der Zwischenergebnisse zurückgeführt.
- **Beispiel:** Direkte Binäre Multiplikation

Die Aufgabe $13 \cdot 11 = 143$ soll in binärer Arithmetik gelöst werden:

$$\begin{array}{r} 1101 \cdot 1011 \\ \hline 1101 \\ 0000 \\ 1101 \\ 1101 \\ \hline 10001111 \text{ Ergebnis} \end{array}$$

Einerkomplement/ Stellenkomplement

- Wird gebildet durch Invertierung aller Stellen
- Zur Darstellung eines negativen Werts geht ein Bit verloren, der Zahlenbereich umfasst nicht das MSB:
 - Most Significant Bit (MSB) 10110110
 - MSB repräsentiert das Vorzeichen: MSB 0 → +, 1 → -

- **Beispiele:**

Die Bitfolge 10000001 würde die negative Dezimalzahl -1 darstellen, da MSB = 1

01010111 87_{dez}

10101000 -87_{dez} (Einerkomplement)

Zweierkomplement

- Ermöglicht die Rückführung der Subtraktion auf die Addition
- In Computern technisch vergleichsweise einfach umzusetzen
- Wird gebildet durch Addition von 1 zum Einerkomplement

- **Beispiel:**

0101 0111 87_{dez}

1010 1000 -87_{dez} (Einerkomplement)

1010 1001 -87_{dez} (Zweierkomplement)

Wir rechnen: 87-87, mit Hilfe der Zweierkomplementmethode:

$$\begin{array}{r} 0101\ 0111 \\ +1010\ 1001 \\ \hline 1111\ 1110 \quad \text{Übertrag} \\ = 0000\ 0000 \quad \text{Ergebnis} \end{array}$$

Übertrag an vorderster Stelle wird gestrichen

Beispiel 1: Subtraktion durch Addition des Zweierkomplements

$$87 - 46 = ?$$

$$46_{\text{dez}} : 0010\ 1110$$

$$\text{Einerkomplement von } 46_{\text{dez}} : 1101\ 0001$$

$$\text{Addition von } 1 \qquad 1$$

$$\text{Zweierkomplement } -46_{\text{dez}} : 1101\ 0010$$

$$87_{\text{dez}} : 0101\ 0111$$

$$\begin{array}{r} \text{Addition} \quad -46_{\text{dez}} : 1101\ 0010 \\ \hline \end{array}$$

$$\begin{array}{r} \text{Übertrag} \quad 1010\ 1100 \end{array}$$

$$\begin{array}{r} \textbf{Ergebnis} \quad 0010\ 1001 \end{array}$$

$$87 - 46 = 41 = 0010\ 1001$$

Anmerkung: Tritt ein Übertrag auf, wird das MSB automatisch 0

Beispiel 2: Subtraktion durch Addition des Zweierkomplements

- negative Ergebnisse

$$2 - 4 = -2$$

4_{dez} : 0000 0100

Einerkomplement von 4_{dez} : 1111 1011

Addition von 1 1

Zweierkomplement -4_{dez} : 1111 1100

2_{dez} : 0000 0010

Addition -4_{dez} : 1111 1100

Ergebnis 1111 1110

Einerkomplement: 0000 0001

Zweierkomplement: 0000 0010

Anmerkung: kein Übertrag bleibt das MSB also 1. Zur Bestimmung des Betrags des Ergebnisses bildet man abermals das Zweierkomplement. Dies liefert dann eine positive Zahl. Das negative Vorzeichen ist ja wegen MSB = 1 bereits bekannt.

Übungen

- Konvertieren Sie nach der Divisionsmethode 285_{dez} in die binäre bzw. hexadezimale Darstellung.
- Addieren Sie die Binärwörter $0101\ 0010 + 0110\ 0111$.
- Subtrahieren Sie die Binärwörter $1010\ 1101 - 0110\ 1011$ (direkte Methode).
- Subtrahieren Sie die Binärwörter $1010\ 1101 - 0110\ 1011$ (Zweierkomplement).
- Wenden Sie die XOR-Operation auf $1011\ 0101 \vee 0110\ 1011$ an.

Informationstheorie - eine kurze Einführung

Claude Shannon, Begründer der Informationstheorie, US-amerikanischer Mathematiker und Elektrotechniker

- „A Mathematical Theory of Communication“, 1949
- Codierung von Information
- Übertragung codierter Information
- Math. Definition von Informationsgehalt
- Definition der Entropie von Information

Nachricht und Information

- Ein Alphabet A besteht aus einer abzählbaren Menge von Zeichen und einer Ordnungsrelation, d. h. eine Regel, durch die eine feste Reihenfolge der Zeichen definiert ist.
- Eine Nachricht ist eine aus den Zeichen eines Alphabets gebildete Zeichenfolge, die von einem Sender (Quelle) ausgehend, in irgendeiner Form einem Empfänger (Senke) übermittelt wird.
- Relevant vor allem in IT und Nachrichtentechnik
- Die Nachrichtenübermittlung erfolgt dabei im technischen Sinne über einen Kanal.
- Umformung der Originalnachricht in das für die Übertragung erforderliche Medium erforderlich, Codierer.
- Empfängerseite: Decodierer wandelt die empfangenen Signale in eine durch den Empfänger lesbare Form.
- Nachrichten lassen sich optimieren bzw. verkürzen .. (In heutiger Zeit u.U. nicht mehr so relevant, wegen Rechnerleistung, Netzwerkbandbreite, ...)



Terminologie für Informationen

- **Zeichen x**: Kleinste Einheit die nicht weiter unterteilt werden kann, z.B. ein einzelner Buchstabe 'A', ein Bit '1', ein Symbol ♥ etc., oder auch landesspezifische Zeichen wie ä, ö, ç
- **Alphabet A**: Definierte endliche Menge von Zeichen, Zeichenvorrat, z.B. lateinisches, kyrillisches, binäres Alphabet $A=\{A..Z\}$, $A=\{A..Z,a..z\}$, $A=\{0,1\}$
- **Nachricht bzw. Wort N**: Eine aus den Zeichen eines Alphabets gebildete Zeichenfolge.
- **Nachrichtenraum $N(A)$ bzw. A^*** : Die Menge aller Nachrichten, die mit den Zeichen eines Alphabets A gebildet werden können.
- **Beispiele: Für Alphabete:**
 1. $\{a, b, c, \dots, z\}$ Die Menge aller Kleinbuchstaben in lexikografischer Ordnung.
 2. $\{0, 1, 2, \dots, 9\}$ Die Menge der ganzen Zahlen 0 bis 9 mit der Ordnungsrelation „<“.
 3. $\{\spadesuit, \heartsuit, \clubsuit, \diamondsuit\}$ Die Menge der Spielkartensymbole in der Reihenfolge ihres Spielwertes.
 4. $\{2, 4, 6, \dots\}$ Die unendliche Menge der geraden natürlichen Zahlen mit Ordnung „<“.
 5. $\{0, 1\}$ Die Binärziffern 0 und 1 mit $0 < 1$.

Interpretation und Information

- Die Extraktion von Information aus einer Nachricht setzt eine Zuordnung (Abbildung) zwischen Nachricht und Information voraus, die Interpretation genannt wird.
- Die Interpretation einer Nachricht ist jedoch nicht unbedingt eindeutig, sondern subjektiv. Das gilt besonders für die Bedeutung, die eine Nachricht tragen kann. Ein und dieselbe Nachricht kann bisweilen auf verschiedene Weisen interpretiert werden.
 - Zum Beispiel kann das Wort „Fuchsschwanz“ dienen. Dabei kann man je nach Kontext an den Schwanz eines Fuchses denken kann, oder aber an eine Säge.
- Die Interpretationsvorschrift muss auch nicht unbedingt offensichtlich sein, Verschlüsselung.
- Die Lehre von der Verschlüsselung von Nachrichten oder Kryptologie entwickelte sich als ein Teilgebiet der Informatik mit großer praktischer Bedeutung.
- „Information“ ist also ein sehr vielschichtiger Begriff, der mathematisch nicht einfach und vor allem auch nicht allgemein in all seinen Facetten fassbar ist.
- Daher sind im Sinne der Informatik Informationen, im Gegensatz zu Nachrichten, nicht exakt definierbare abstrakte Objekte.

Codierung von Buchstaben, Ziffern und Sonderzeichen

- Für die Kodierung lateinischer Zeichen wird fast nur noch bei Großrechnern die 8-Bit-Kodierung **EBCDIC** verwendet, die IBM parallel zu ASCII für sein System/360 entwickelte.
- Der **American Standard Code for Information Interchange (ASCII)**, ab 1963, ist eine 7-Bit-Zeichenkodierung. Sie dient als Grundlage für spätere, auf mehr Bits basierende Kodierungen für Zeichensätze.
- Die Zeichenkodierung definiert 128 Zeichen, die Zeichen 0 bis 32 sind nicht druckbare Sonderzeichen gefolgt von 95 druckbaren Zeichen, beginnend mit dem Leerzeichen.
- Druckbare Zeichen: lateinisches Alphabet in Groß- und Kleinschreibung, die zehn arabischen Ziffern sowie einige Interpunktionszeichen, wie Satzzeichen und andere Sonderzeichen.
- Die wichtigsten Sonderzeichen sind Zeilenvorschub (Line Feed, LF), Wagenrücklauf (Carriage Return, CR), Seitenvorschub (Form Feed, FF), Rückwärtsschritt (Backspace, BS), horizontaler und vertikaler Tabulator (HT und VT), Eingabe löschen (Escape, ESC).
- Die nicht druckbaren Steuerzeichen enthalten Ausgabezeichen wie Zeilenvorschub oder Tab, Übertragungsende, etc.
- Heute wird fast immer zur Erweiterung von ASCII ein 8-Bit-Code verwendet, 256 Zeichen.
- Auch 8-Bit-Codes bieten zu wenig Platz, um alle Zeichen, die gebraucht werden, gleichzeitig unterzubringen. Es wurden **Zeichensatztabellen** definiert, die den darstellbaren Zeichen und Steuerzeichen Zahlenwerte zuordnen, auch Codetabellen oder Codepages. Für die Darstellung von Text auf Computern sind üblicherweise also sowohl eine Zeichensatztabelle wie auch ein Font notwendig.
- Beispiele für Codepages: <https://de.wikipedia.org/wiki/Zeichensatztabelle>.

Beispiel erweiterter ASCII-Code für die Codepage 850

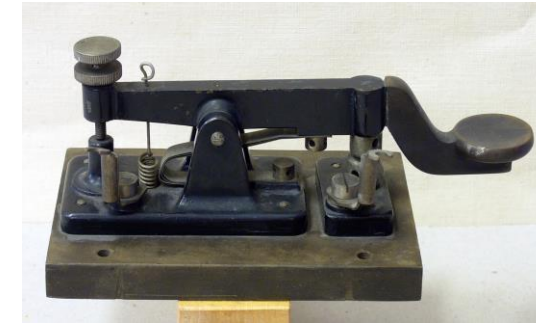
000	NUL	033	!	066	B	099	c	132	ä	165	ñ	198	ã	231	þ
001	Start Of Header	034	"	067	C	100	d	133	å	166	²	199	Ä	232	ð
002	Start Of Text	035	#	068	D	101	e	134	å	167	³	200	Å	233	ú
003	End Of Text	036	\$	069	E	102	f	135	ç	168	¸	201	Æ	234	û
004	End Of Transmission	037	%	070	F	103	g	136	è	169	©	202	Ⓐ	235	ü
005	Enquiry	038	&	071	G	104	h	137	é	170	ª	203	Ⓑ	236	ý
006	Acknowledge	039		072	H	105	i	138	ê	171	½	204	Ⓒ	237	ÿ
007	Bell	040	(	073	I	106	j	139	ï	172	¾	205	Ⓓ	238	—
008	Backspace	041	)	074	J	107	k	140	î	173	¿	206	Ⓔ	239	‘
009	Horizontal Tab	042	*	075	K	108	l	141	ì	174	«	207	Ⓕ	240	-
010	Line Feed	043	+	076	L	109	m	142	Ä	175	»	208	Ⓖ	241	±
011	Vertical Tab	044	,	077	M	110	n	143	Å	176	○	209	Ⓗ	242	_
012	Form Feed	045	-	078	N	111	o	144	É	177	◌	210	Ⓘ	243	¼
013	Carriage Return	046	.	079	O	112	p	145	Ê	178	◌	211	Ⓚ	244	½
014	Shift Out	047	/	080	P	113	q	146	Ë	179		212	Ⓛ	245	¾
015	Shift In	048	0	081	Q	114	r	147	Ì	180	¡	213	Ⓜ	246	÷
016	Delete	049	1	082	R	115	s	148	Í	181	À	214	Ⓝ	247	ˆ
017	-- frei --	050	2	083	S	116	t	149	Î	182	Á	215	Ⓖ	248	˜
018	-- frei --	051	3	084	T	117	u	150	Ï	183	Â	216	Ⓗ	249	˘
019	-- frei --	052	4	085	U	118	v	151	Ù	184	Ã	217	Ⓜ	250	˙
020	-- frei --	053	5	086	V	119	w	152	Ú	185	Ä	218	Ⓝ	251	˚
021	Negative Acknowledge	054	6	087	W	120	x	153	Û	186	Å	219	Ⓖ	252	¸
022	Synchronous Idle	055	7	088	X	121	y	154	Ü	187	Æ	220	Ⓗ	253	ˆ
023	End Of Transmission Block	056	8	089	Y	122	z	155	Ý	188	Ç	221	Ⓝ	254	˜
024	Cancel	057	9	090	Z	123	{	156	ÿ	189	È	222	Ⓝ	255	
025	End Of Medium	058	:	091	[	124		157	ÿ	190	É	223	Ⓝ		
026	Substitute	059	;	092	\	125	}	158	ÿ	191	Ê	224	Ⓝ		
027	Escape	060	<	093	]	126	~	159	ÿ	192	Ë	225	Ⓝ		
028	File Separator	061	=	094	^	127	¸	160	á	193	Ⓐ	226	Ⓝ		
029	Group Separator	062	>	095	_	128	Ç	161	í	194	Ⓑ	227	Ⓝ		
030	Record Separator	063	?	096	`	129	ü	162	ó	195	Ⓒ	228	Ⓝ		
031	Unit Separator	064	@	097	a	130	é	163	ú	196	—	229	Ⓝ		
032		065	A	098	b	131	ä	164	ñ	197	+	230	μ		

Der Unicode

- Der Unicode ist eine Erweiterung des ASCII-Codes, mit dem sämtliche Zeichensysteme in den bekannten Schriftkulturen ausgedrückt werden können (Erste Version 1991). <https://unicode.org/main.html>.
- Unicode als internationalen Standard (ISO 10646) und integriert in Version 12.0 im März 2019 inzwischen 150 Schriftsysteme mit 137292 Zeichen und Emoji.
- ‚Alle‘ modernen Betriebssysteme, Programmiersprachen und Anwendungen basieren inzwischen auf dem Unicode-System. Dazu gehören zahlreiche Sonderzeichen, mathematische Formelzeichen, fernöstliche Symbole und Angaben über die Schreibrichtung und Stellung der Zeichen. Details findet man auf der Unicode-Webseite 2.
- Die Umsetzung des Unicode-Zeichensatzes in konkrete Codierungen erfolgt mit dem Unicode Transformation Format (UTF). Dieses existiert in verschiedenen Ausprägungen.
- UTF-8 verwendet zur Codierung zwischen 1 und 4 Byte. Vorteil, dass die 1-Byte Codierung mit dem 127-Bit ASCII Code übereinstimmt und ist bei Verwendung von lateinischen Zeichen sehr effizient (im Vergleich zu 2 Byte bei UTF-16).
- UTF-32 verwendet beispielsweise immer genau 4 Byte, viel Speicherplatz ist erforderlich (vgl. 1 Byte bei ASCII)
- UTF-16 benötigt pro Zeichen 2 Byte (für die „gebräuchlichen“ Symbole) oder 4 Byte (für die „exotischeren“ Symbole).
- flexiblere Varianten UTF-16 (Windows, OS-X) und UTF-8 (WWW, Email, Unix) sind weiter verbreitet.
- **Erst durch neuere Speicher- und Rechnertechnologien möglich, weil mehr Speicherplatz gebraucht wird.**

Weitere bekannte Codierungen, Beispiele

- Morse–Codierung (Samuel Morse (1791 - 1872), war ein gebräuchlicher Code zur telegrafischen Übermittlung von Buchstaben, Ziffern und weiterer Zeichen. Wird heute noch in der Luft- und Schifffahrt verwendet, ansonsten kaum noch Bedeutung.
- Als Strichcode, oder Barcode wird eine optoelektronisch lesbare Schrift (Abbildung von Daten in binären Symbolen) bezeichnet, die aus verschiedenen breiten, parallelen Strichen und Lücken besteht. in den 1970er Jahren begann die Verbreitung in Supermarktketten.
- Der QR-Code (englisch Quick Response) besteht aus einer quadratischen Matrix aus schwarzen und weißen Quadraten, die die kodierte Daten binär darstellen (Firma Denso Wave, 1994). entwickelt wurde. Die Daten im QR-Code sind durch einen fehlerkorrigierenden Code erweitert. Dadurch wird der Verlust von bis zu 30 % des Codes toleriert, d. h., er kann auch dann noch dekodiert werden. Die Verwendung des QR-Codes ist lizenz- und kostenfrei.



Von Hannes Grobe (talk) - Schulhistorische Sammlung Bremerhaven, CC BY 3.0, <https://commons.wikimedia.org/w/index.php?curid=4626456>



Multimediale Daten

- Grafiken, Bilder
 - Rastergrafik, NetPBM-Format (Network Portable BitMap)
 - Grafik wird aus einer Folge einzelner Rasterpunkte dargestellt und für jeden Punkt einzeln der Farbwert und eventuelle Transparenz definiert (abhängig von der Farbtiefe)
 - Ein Format für Vektorgrafik ist das SVG-Format für Scalable Vector Graphics. Hier können mit allgemeinen Pfaden, Formen und Texten samt weiterführenden Transformationen und Effekten als Text oder mit Programmen erfasst werden.
- Audio, wie WAVE, FLAC-Format (Free Lossless Audio Codec) und MP-3
- Videodaten sind typischerweise eine Kombination von bewegtem Bild und Tondaten. Einzelbildspeicherung bei Motion JPEG, Animated Portable Network Graphics oder bei Animated GIF-Format. Videoformate wie MP4 speichern nur wenige Bilder komplett ab, und versuchen durch Speichern von Unterschieden wiederum kombiniert mit einer Reduktion von für Menschen unwichtigen Informationen den Speicherbedarf deutlich zu reduzieren.



Anwendungsbereiche von Codierung bzw. Codes

- Physische Übertragung von Information
 - Leitungscodierung, Kanalcodierung
 - Fehlerkorrigierende Übertragung von Information
 - (Effiziente) Persistierung von Information
 - Dateisysteme, Kompressionsverfahren
 - Verschlüsselung, sym.-/ asymmetrische Verfahren

Codesicherung

- **Problem:** Störungen können dazu führen, dass einzelne Bits nicht richtig übertragen werden (0 statt 1/ 1 statt 0).
- **Lösung:** Durch Hinzufügen zusätzlicher Bits bzw. geschickter Auswahl der Codeworte kann die Störanfälligkeit eines Codes erhöht werden, so dass eine Fehlererkennung bzw. eine Fehlerkorrektur möglich ist.

Grössenordnungen

$$1\text{K} = 1024 = 2^{10} \text{ (K = kilo)}$$

$$1\text{M} = 1024 \cdot 1024 = 2^{20} \text{ (M = mega)}$$

$$1\text{G} = 1024 \cdot 1024 \cdot 1024 = 2^{30} \text{ (G = giga)}$$

$$1\text{T} = 1024 \cdot 1024 \cdot 1024 \cdot 1024 = 2^{40} \text{ (T = tera)}$$

$$1\text{P} = 1024 \cdot 1024 \cdot 1024 \cdot 1024 \cdot 1024 = 2^{50} \text{ (P = Peta)}$$

$$1\text{E} = 1024 \cdot 1024 \cdot 1024 \cdot 1024 \cdot 1024 \cdot 1024 = 2^{60} \text{ (E = exa)}$$

Für Längen, Zeiten:

$$1\text{m} = 10^{-3} \text{ (m = milli)}$$

$$1\mu = 10^{-6} \text{ (\mu = mikro)}$$

$$1\text{n} = 10^{-9} \text{ (n = nano)}$$

$$1\text{p} = 10^{-12} \text{ (p = pico)}$$

$$1\text{f} = 10^{-15} \text{ (f = femto)}$$

Datentypen und Wertebereiche von Zahlen

Typ	Länge		Max. Wert
Unsigned short	16 Bit		0 – 65 536
Short	16 Bit	1 Vorzeichenbit	-32 768 to 32 767
Unsigned int	32 Bit		0 to 4,294,967,295
Int	32 Bit	1 Vorzeichenbit	-2,147,483,648 to 2,147,483,647
Unsigned long	64 Bit		0 to 1844,6744,0737,0955,1615
Long	64 Bit	1 Vorzeichenbit	922,3372,0368,5477,5808 to 922,3372,0368,5477,5807
Unsigned char	1 Byte		0 to 255
Char	1 Byte	1 Vorzeichenbit	-128 to 127
Float	32 Bit	6 Dezimalstellen	1.2E-38 to 3.4E+38
Double	64 Bit	15 Dezimalstellen	2.3E-308 to 1.7E+308

- Short und Int hängen von der Wortgröße ab, je nach Hardware Architektur können das 8, 16, 32 oder 64 Bit sein.
- Je nach Hardware Architektur und Programmiersprache können kleine Unterschiede existieren.